

Assignment 6 - DDL: Database Modelling

Roles

Customer: Michel Weingarten

Designer: Eran Golani

Reviewer: Barak Blaish

Main Use Case (written by customer)

Lemon Scooter Rentals is an urban electric scooter sharing service. Customers use a mobile application to locate nearby scooters, unlock them with a QR scan or app button, ride to their destination, and park the scooter in a permitted zone. Payment is charged automatically to the customer's registered payment method at the end of each trip. The operations team must track the real-time status and location of every physical scooter, schedule maintenance when battery levels drop or when damage is reported and ensure that pricing is calculated correctly for each rental. Customer service staff need to look up rental and payment history quickly in case of disputes.

Customer Requirements

The following requirements were defined by the customer and represent the core needs of the business:

R1 - Customer management: The system must store customer details including name, contact information, date of birth, and driving license number. Each customer must be uniquely identifiable.

R2 - Scooter inventory: The system must track individual physical scooter units, each identified by a unique serial number. Each unit is associated with a scooter model that describes technical specifications such as maximum speed and battery range.

R3 - Scooter availability: The system must know whether any individual scooter is available, currently rented, under maintenance, or damaged. A scooter must not be assignable to a new rental while its status is not 'available'.

R4 - Rental orders: The system must record each trip with its customer, scooter, start time, end time, pickup location, return location, and distance travelled.

R5 - Pricing and payments: The system must record a payment event per rental and be able to charge a customer's payment info.

R6 - Maintenance tracking: Each maintenance event on a physical scooter must be recorded with its type, description, start date, and end date so the operations team can audit service history.

Designer's Understanding of the Use Case

As the designer, I understand that the core challenge in this system is maintaining a reliable and always-accurate picture of the state of every physical scooter. Unlike a traditional rental business that manages quantities, Lemon must track every individual unit because the scooter's location, battery level, and condition are all operationally critical and unique per unit.

I understand that rental availability is determined by the status field on the individual scooter record. This is a simpler and more accurate model than computing availability from overlapping rental date ranges (as is done in a hotel or equipment-sharing system), because a scooter can only be rented to one customer at a time and its status can change for reasons other than an active rental (maintenance, damage).

Finally, I understand that pricing and payment are coupled to the rental itself: every rental needs to end with a payment taken from the payment info of the customer that is saved.

Discussion 1 - Individual Unit Tracking vs. Quantity-Based Tracking

Customer: I suggested that since all scooters of the same model behave the same way, we could simply track how many units of each model are available, similar to how a bike rental counts bikes of a given type.

Designer: I explained that electric scooters have unique operational states: each has its own battery level, GPS location, and service history. If we only tracked quantities, we would not be able to dispatch the closest scooter to a customer, schedule maintenance for a specific unit, or assign liability for damage to the customer who last rented it.

Reviewer: I agreed that individual tracking is necessary. I added that separating the model specification table from the individual unit table is the correct normalization choice, because model data (speed, range) is shared across many units and should not be duplicated.

Final Decision: We agreed to track each scooter as an individual row in the scooters table, with a foreign key reference to a scooter_models table for shared technical specifications.

Discussion 2 - Availability Tracking

Customer: I asked whether availability should be computed by looking at overlapping rental date ranges, or by storing a simple status field.

Designer: I explained that for a scooter fleet, a status field is more appropriate. A scooter becomes unavailable for reasons beyond active rentals: it may be damaged, its battery may be too low to rent, or it may be in a maintenance shop. A date-range availability model would not capture these states. The status field on the scooters table covers all cases with a single lookup.

Reviewer: I agreed and noted that the allowed status values should be clearly defined in the schema documentation to prevent inconsistent data entry.

Final Decision: We agreed to use a status field on the scooters table with defined allowed values: 'available', 'rented', 'maintenance', 'damaged', 'retired'.

Discussion 3 - Separating Maintenance from Incidents

Customer: I initially suggested having a single 'issues' table for both maintenance events and damage incidents, since both result in a scooter being taken offline.

Designer: I explained that maintenance and incidents are fundamentally different entities. Maintenance is a scheduled operational activity initiated by staff. An incident is an unexpected event during a rental that may trigger a charge to the customer. Mixing them in one table would require many nullable columns and complicate queries for billing, operations, and reporting.

Reviewer: I agreed that separation is correct. I added that the incidents table should reference both the rental and the scooter, because some incidents (e.g., theft after return) occur outside an active rental and the rental reference would then be null.

Final Decision: We agreed on two separate tables: maintenance_records (linked to scooter only) and incidents (linked to both rental and scooter, with rental being nullable for post-return events).

Reviewer Summary

As the reviewer, I verified that the final database design fits the customer requirements and follows normalization principles.

The main improvements included separating the scooter units table from the scooter models table to avoid duplicated specifications, using a status field instead of date-range overlap for availability tracking, and separating maintenance records from incident reports into two dedicated tables to reduce nullable columns and simplify billing queries.

Overall, the final database design provides an organized and reliable solution for managing Lemon Scooter Rentals operation.

Final Review Summary

The final database includes seven tables connected through primary and foreign keys. The tables are organized around the core business entities: customers, scooter inventory, rentals, payments, maintenance, and incidents.

The design has: scooter availability checks using a status field, individual unit tracking with shared model specifications, payment charging using stored customer payment info, maintenance history per scooter, and incident reporting linked to rentals.

See the following schema for full design:

